



INDIVIDUAL ASSIGNMENT

CT128-3-2-ISC

IMPLEMENTATION OF SECURE SYSTEM

APU2F2411CS(CYB) - APD2F2411CS(CYB)

HAND OUT DATE: 14th MAY 2025

HAND IN DATE: 8th AUGUST 2025

WEIGHTAGE: 40%

INSTRUCTIONS TO CANDIDATES:

- 1. Assignment is to be submitted through online submission (Moodle).**
- 2. Students are advised to underpin their answers with the use of references
(cited using the APA Name System of Referencing)**
- 3. Late Submission will be awarded zero (0) unless Extenuating Circumstances
(EC) are upheld.**
- 4. The case of plagiarism will be penalized.**
- 5. You must obtain 50% overall to pass this module.**

MODULE NAME:	IMPLEMENTATION OF SECURE SYSTEM
MODULE INTAKE CODE:	CT128-3-2-ISC
LECTURER NAME:	DR. KAMALAKANNAN MACHAP
TP NUMBER:	TP070844
STUDENT NAME:	HEMANTHARAJAH A/L N KRISHNARAJAH

Contents

1.0 Introduction	4
1.1 General Introduction to Network Security	4
1.2 Overview of Intrusion Detection Systems (IDS).....	5
1.3 Recent Issues and Challenges.....	6
2.0 Vulnerabilities and Potential Attacks	7
3.0 Best Practices and Techniques	9
4.0 Implementation.....	14
4.1 Linux System Update and Upgrade Process.....	14
4.2 Suricata Installation on Kali Linux.....	15
4.3 Python Script for Initiating a Ping Flood (Attack Simulation).....	16
4.4 Configuring Suricata Rules for Ping Flood Detection (Defense Setup).....	17
4.5 Testing and Activating Suricata	20
4.6 Initiating the Ping Flood (Attack Simulation for Detection).....	21
4.7 Successful Ping Flood Detection by Suricata.....	22
5.0 Conclusion.....	23
6.0 References	24

1.0 Introduction

1.1 General Introduction to Network Security



Figure 1: Network Security

Network security is a broad term encompassing technologies, processes, and policies that protect networks and data from breaches, misuse, or unauthorized access. With the rapid growth of cloud computing, mobile devices, and online services, networks are constantly exposed to evolving cyber threats. Modern enterprises face threats ranging from phishing and malware to advanced persistent threats (APTs) and zero-day vulnerabilities (Cisco, 2025). According to Fortinet's 2025 Global Threat Landscape Report, attackers now leverage automation and AI to execute faster and more sophisticated attacks. Notably, there was a 42% rise in stolen credentials and 97 billion exploitation attempts recorded globally (Fortinet, 2025).

In this environment, securing networks is not optional but essential. A single breach can result in data loss, financial damage, and reputational harm. Legal frameworks like the

GDPR and Malaysia's PDPA mandate robust cybersecurity measures (European Commission, 2018). Consequently, organizations deploy various security layers such as firewalls, antivirus software, encryption, and more. Among these, Intrusion Detection Systems (IDS) play a vital role by monitoring traffic and identifying suspicious patterns, often acting as the second line of defense after a firewall (IBM Security, 2023).

1.2 Overview of Intrusion Detection Systems (IDS)



Figure 2: Suricata Logo

Intrusion Detection Systems (IDS) are programs or tools used to check network traffic for harmful activities or violations of security rules. There are two main types: host-based (HIDS) and network-based (NIDS). HIDS checks what's happening on individual

devices, while NIDS, like Suricata, monitors the overall traffic moving between devices (Pluralsight, 2023).

Suricata is a fast, open-source IDS and IPS created by the Open Information Security Foundation (OISF). It can deeply inspect data packets, send real-time alerts, pull out files, and recognize different protocols. Suricata works by matching traffic against preset rules (usually ones that work with Snort), helping it detect unusual behavior, attacks, and suspicious data (OISF, 2024).

Compared to older IDS systems, Suricata can run multiple tasks at once (multi-threading), understands protocols better, and supports JSON output. It's able to manage large amounts of data quickly, which is useful for modern companies and government systems. With regular rule updates from Emerging Threats and support for tools like Kibana and Zeek, Suricata gives a full view of both regular and encrypted traffic (Ali et al., 2021).

1.3 Recent Issues and Challenges

Even though IDS tools like Suricata are important, they still face several challenges. One major problem is false positives when safe network activity is wrongly flagged as dangerous. Too many false positives can overwhelm security teams and make people lose confidence in the system (Ali et al., 2021). On the other hand, false negatives when real threats are missed can lead to serious damage.

Another big challenge is performance issues. Suricata uses a lot of system resources, especially on networks with heavy traffic. If it's not set up properly or uses old rules, it might slow down or fail to catch important threats. Encrypted traffic, like HTTPS, also creates problems. While Suricata can recognize SSL handshakes and some protocols, it can't fully inspect the content unless the traffic is decrypted. This raises privacy concerns and may require complicated systems (Santos et al., 2022).

IDS tools are also at risk from tricks used by advanced hackers. These attackers can split up data, change it, or hide it using tunnels to avoid being detected. Also, if security policies are incorrect or not updated often, networks become easier targets. A report by IBM in 2024 showed that old detection rules and bad configurations caused 27% of data breaches that year (IBM, 2024).

2.0 Vulnerabilities and Potential Attacks

Although Suricata and other Intrusion Detection Systems (IDS) provide crucial defense against cyber threats, they do have several shortcomings. Their effectiveness can be compromised or exploited by malicious actors due to these flaws. In order to construct a trustworthy and safe cybersecurity framework, it is critical to identify and resolve these restrictions.

The fact that IDS often generate false positives and false negatives is one of its biggest drawbacks. Overwhelming analysts with unnecessary notifications and producing alert fatigue, false positives happen when benign activity is incorrectly marked as suspicious. If there is an overwhelming number of false alarms, it may be difficult to respond quickly enough to actual dangers. False negatives, on the other hand, let legitimate dangers pass undiscovered, giving hackers a free pass to cause havoc. More than 60% of firms using network-based intrusion detection systems had operational disruptions because of alert fatigue, and 22% had at least one undiscovered breach in the previous year owing to a false negative (SANS Institute, 2023). When users lose faith in the system because of these errors, incident response becomes much more difficult.

Performance concerns also pose a serious risk, particularly during periods of excessive traffic or DDoS attacks. Although Suricata is designed for high performance and supports multithreading, its effectiveness depends on adequate hardware provisioning and configuration. During peak traffic, Suricata may discard packets when deployed on systems with insufficient resources or incorrectly configured parameters, which might result in missed detections (OISF, 2024). Without monitoring indicators like packet loss

rates or CPU utilization, managers might not notice these issues until after the damage is done.

Another key difficulty includes encrypted traffic. As more than 85% of worldwide internet data is encrypted via HTTPS or TLS (Cisco, 2025), IDS systems like Suricata lose visibility into actual packet contents. While Suricata can distinguish SSL/TLS handshakes and encryption use, it cannot investigate payloads unless combined with an SSL decryption proxy or man-in-the-middle arrangement. Cybercriminals exploit this blind spot by hiding threats or data exfiltration inside encrypted connections. Without integration with SSL inspection tools, Suricata can only collect metadata such as IP addresses or digital certificates, which is typically inadequate to detect sophisticated threats (Santos, Moura, & Oliveira, 2022).

Attackers also regularly utilize fragmentation and evasion strategies to outmaneuver IDS systems. Though Suricata enables stream reassembly and protocol normalization, these functionalities must be adequately enabled. If not, adversaries can create fragmented packets or overlapping IP pieces that avoid detection. This is especially harmful in networks with old settings or poor packet processing. Research by Korczynski et al. (2020) indicated that many systems lack effective source address filtering, making them vulnerable to faked traffic and evasive tactics that exploit IDS shortcomings (Korczynski et al., 2020).

Suricata's reliance on signature-based detection is another area of concern. It operates by matching traffic against predetermined rules, which is effective for known threats but useless against novel attack vectors such as zero-day exploits or advanced persistent threats (APTs). When rule sets are obsolete or incomplete, attackers can easily slip through undetected. Despite the need of upgrading these signatures such as those offered by Emerging Threats many organizations fail to do so consistently. This indicated that 27% of successful breaches were linked to misconfigured or obsolete IDS rules (IBM Security, 2024).

Alert overload is another difficulty that comes when Suricata isn't properly configured. In high-traffic scenarios, the system may generate thousands of notifications each hour, many of which are meaningless. Without prioritization, filtering, or correlation, security professionals may find it difficult to spot serious risks within the noise. This problem becomes worse in networks without centralized logging or SIEM integration, where data is distributed and analysis becomes more challenging.

Finally, human factors and configuration mistakes are chronic weaknesses. mistakes such as selecting the wrong monitoring interface, implementing faulty custom rules, or mishandling complex features like flowbits or thresholds can substantially hamper detection capabilities. For instance, if Suricata runs in promiscuous mode on an unsuitable interface or if memory buffers are insufficiently big, it may fail to collect whole traffic flows. Even skilled workers are prone to such errors without sufficient documentation and regular configuration testing (Ali, Habib, Ullah, & Islam, 2021).

In summary, although Suricata is a robust and adaptive IDS, its effectiveness is heavily dependant on precise configuration, regular updates, and integration with broader security architecture. Limitations such as alert overload, encrypted traffic invisibility, performance degradation, reliance on static signatures, and setup problems might compromise its usefulness. Addressing these through proactive monitoring, constant tuning, and layered protections is vital to maintaining a solid cybersecurity posture.

3.0 Best Practices and Techniques

To guarantee Intrusion Detection Systems (IDS) are effective in detecting, preventing, and responding to cyber threats, organizations must implement a number of coordinated best practices. These solutions try to maximize detection accuracy, limit false positives, support system performance, and enable quick reaction in dynamic network conditions.

1. Deploy the IDS at Key Network Locations

Effective deployment begins with strategic placement of sensors. Network-based IDS should be installed at traffic aggregation points such as between internal segments and the internet gateway or between subnetworks handling sensitive data. This increases the chance of detecting lateral movement or external attacks. A 2022 study shows that improper sensor placement led to delayed threat detection in over 35% of simulated breach scenarios (Ali et al., 2022).

2. Enable Promiscuous Mode on Monitoring Interfaces

For IDS to inspect all network traffic, the interfaces must operate in promiscuous mode. This allows the IDS to receive every packet on the network segment, not just traffic addressed to it. Without this, visibility is limited, and stealthy attacks such as ARP spoofing or unauthorized scanning may go undetected (Mahmood et al., 2022). Promiscuous mode is particularly critical for detecting threats in flat or unsegmented networks.

3. Configure the IDS for Real-Time Monitoring and Alerting

Real-time threat detection enables faster response and mitigation. IDS must be continuously active, using live packet analysis and preconfigured alert thresholds. Modern systems support automatic notification via email, syslog, or integration with a Security Information and Event Management (SIEM) platform. Real-time monitoring has been linked to 40% faster mean time to detect (MTTD) in incident response (IBM Security, 2023).

4. Update Detection Rules and Threat Feeds Regularly

Keeping detection signatures up to date is vital for identifying current threats. IDS tools typically rely on open-source or commercial rule sets that are updated to reflect new vulnerabilities and attack patterns. Regular updates automated, if possible, ensure the IDS does not miss emerging threats. A recent survey by CyberEdge (2024) found that 68% of breaches exploited outdated or unpatched detection logic.

5. Monitor and Analyze Alerts and Logs Proactively

An IDS is only effective if its output is reviewed regularly. Security analysts must monitor alerts for anomalies and correlate them with system logs or endpoint data. Tools such as Splunk, Kibana, and Wazuh can help visualize and filter alerts efficiently. Frequent log reviews reduce the risk of missing persistent threats and improve incident handling (Santos et al., 2022).

6. Tune IDS Parameters for Network Performance

IDS systems must be optimized to match the network size and traffic volume. This includes adjusting buffer sizes, packet capture filters, and alert thresholds. Improper tuning can result in dropped packets, excessive false positives, or system overload. Adaptive tuning based on traffic baselining has been shown to improve detection accuracy by up to 23% in enterprise networks (Khan et al., 2023).

7. Develop Custom Detection Rules Based on Organizational Needs

Public rules are useful, but they may not cover unique organizational risks. Writing custom IDS rules allows detection of internal policy violations, misuse of proprietary services, or specific compliance violations. Custom rule development is encouraged in critical environments such as healthcare, finance, or government, where context-specific threats are more prevalent (Choudhury & Lee, 2023).

8. Validate Detection Accuracy Using Test Traffic or PCAPs

Testing the IDS using known attack traffic or .pcap files allows analysts to validate rule effectiveness without impacting live systems. This practice is essential for quality assurance and helps confirm that the IDS responds correctly to specific scenarios. It also supports training and benchmarking (Santos et al., 2022).

9. Integrate IDS with Centralized Logging and SIEM Solutions

Integration with platforms like ELK Stack, Wazuh, or Microsoft Sentinel enables comprehensive analysis of alerts, system logs, and user behaviors. This supports threat correlation, automated response, and compliance reporting. According to a 2023 Gartner report, organizations with IDS-SIEM integration reduced average breach containment time by 38% (Gartner, 2023).

10. Automate Rule Updates and Audit Configurations Regularly

Automation of routine tasks such as rule updates, alert forwarding, and system health checks increases operational efficiency. Additionally, regular audits of rule effectiveness, false positive rates, and system performance help maintain the IDS's effectiveness over time. Organizations that perform quarterly IDS audits report lower false alert volumes and higher incident response rates (CyberEdge, 2024).

11. Use IDS as Part of a Layered Security Strategy

IDS should not function as a standalone tool. It is most effective when part of a multi-layered defense model that includes firewalls, endpoint detection and response (EDR), user behavior analytics, and secure configuration management. In this model, the

IDS provides deep network visibility, alerting anomalies that may bypass other controls (Mahmood et al., 2022). The combination enhances security posture and reduces single points of failure.

4.0 Implementation

4.1 Linux System Update and Upgrade Process

```
(kali@hemanth)-[~]
$ sudo apt update -y
Get:1 http://kali.download/kali kali-rolling InRelease [41.5 kB]
Get:2 http://kali.download/kali kali-rolling/main amd64 Packages [21.0 MB]
Get:3 http://kali.download/kali kali-rolling/main amd64 Contents (deb) [51.4 MB]
Ign:3 http://kali.download/kali kali-rolling/main amd64 Contents (deb)
Get:4 http://kali.download/kali kali-rolling/contrib amd64 Packages [117 kB]
Get:5 http://kali.download/kali kali-rolling/contrib amd64 Contents (deb) [327 kB]
Get:6 http://kali.download/kali kali-rolling/non-free amd64 Packages [198 kB]
Get:7 http://kali.download/kali kali-rolling/non-free amd64 Contents (deb) [911 kB]
Get:3 http://mirror.aktkn.sg/kali kali-rolling/main amd64 Contents (deb) [51.4 MB]
Fetched 74.0 MB in 18s (4,133 kB/s)
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
2426 packages can be upgraded. Run 'apt list --upgradable' to see them.
```

Figure 3: sudo apt update -y

```
(kali@hemanth)-[~]
$ sudo apt upgrade -y
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
Calculating upgrade... Done
The following packages were automatica
```

Figure 4: sudo apt upgrade -y

First, I worked on updating my Kali Linux system. I ran the `sudo apt update` command to refresh the list of available packages. I then proceeded to execute `sudo apt upgrade -y`. This command was meant to install any available updates for my installed packages, with the `-y` flag automatically confirming all prompts.

4.2 Suricata Installation on Kali Linux

```
(kali@hemanth)-[~]
$ sudo apt install suricata -y
[sudo] password for kali:
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
The following packages were automatically installed and are no longer required:
catfish debtags dh-elpa-helper fonts-roboto-slab gfortran-11 gir1.2-xfconf-0 idle-python3.9
kali-debtags libatk1.0-data libavfilter7 libavformat58 libbfi01
libcmark-gfm-extensions0.29.0.gfm.3 libcmark-gfm0.29.0.gfm.3 libev4 libexporter-tiny-perl
libfile-find-rule-perl libflac8 libfmt8 libgeos3.10.2 libgfortran-11-dev libgphoto2-l10n
libgraphblas6 libgssdp-1.2-0 libgupnp-1.2-1 libhawtjni-runtime-java libhttp-server-simple-perl
libice-dev libilmbase25 libinchi1 libjansi-native-java liblist-moreutils-perl
liblist-moreutils-xs-perl libllvm12 liblttng-ust-ctl4 liblttng-ust0 libmms0 libncurses5
libnginx-mod-http-geoip libnginx-mod-http-image-filter libnginx-mod-http-xslt-filter
libnginx-mod-mail libnginx-mod-stream libnginx-mod-stream-geoip libnsl-dev
libnumber-compare-perl libofa0 libopenexr25 libperl5.32 libperl5.34 libpostproc55
libprimesieve9 libpython3.9-dev libsm-dev libsrtp1.4-gnutls libswscale5 libtbb2 libtexlua53
libtext-glob-perl libtiffxx5 libtinfo5 libtirpc-dev libunicode-escape-perl libwebsockets16
```

Figure 5: sudo apt install suricata -y

After that, I continued installing Suricata on my Kali Linux machine. I initiated the installation using the command `sudo apt install suricata -y`. This command uses apt, Kali's package manager, to download and install Suricata and its necessary dependencies.

4.3 Python Script for Initiating a Ping Flood (Attack Simulation)

```
2  from pythonping import ping
3
4  # Get ip address or hostname
5  host_address = input("Enter single IP address or hostname: ")
6  while True:
7      try:
8          # Ping host
9          result = ping(
10             host_address,
11             count=10000,
12             size=1000,
13             timeout=1
14         )
15
16         print(result)
17
18     except KeyboardInterrupt:
19         print("\n Ping flood stopped by user.")
20         break
21
22     except Exception as e:
23         print(f"\n Error: {e}")
24         break
25
26  input("\n Press Enter to continue . . .CTRL-c to stop.")
```

Figure 6: ping_flood.py

This is a Python script I created, which I designed to imitate a "ping flood" attack. This script employs the pythonping module to send an unusually high volume of ICMP echo request packets to a target IP address or hostname that I specify. I've configured it to send 10,000 packets of 1000 bytes each, with a 1-second timeout per packet, looping continually. This destructive arrangement is specifically meant to create enough network traffic to trigger an Intrusion Detection System (IDS) when configured correctly. I've also incorporated simple error handling to stop the flood if I interrupt it or if any other unanticipated complications emerge.

4.4 Configuring Suricata Rules for Ping Flood Detection (Defense Setup)

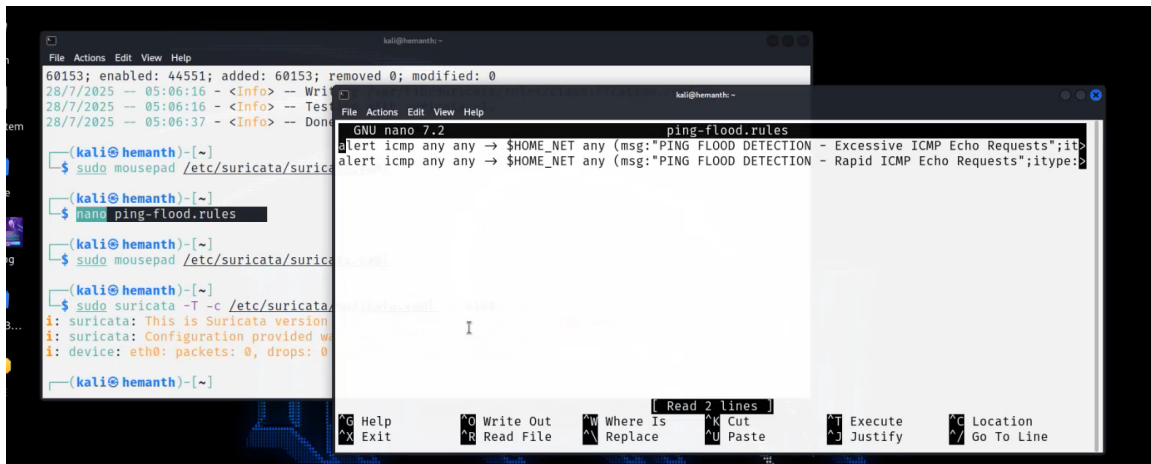


Figure 7: ping-flood.rules

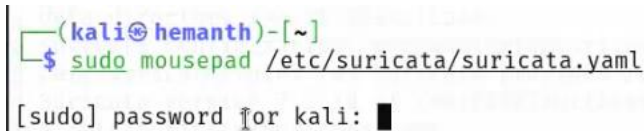
Following the attack simulation script, I moved on to the defensive side by configuring Suricata to detect this very type of activity. I opened a new rule file, `ping-flood.rules`, located within my kali home directory (`/home/kali/`). Inside this file, I added two specific Suricata rules designed to identify excessive and rapid ICMP echo requests, which are characteristic of a ping flood:

1. **Excessive ICMP Echo Requests:** This rule (`alert icmp any any -> $HOME_NET any (msg:"PING FLOOD DETECTION - Excessive ICMP Echo Requests";ittype:8;flow:to_server;threshold: type limit, track by_src, count 100, seconds 10;classtype:attempted-dos;sid:21;))` is set to alert if it detects more than 100 ICMP echo requests (type 8) originating from any source to my `$HOME_NET` (internal network) within a 10-second window. It tracks these requests per source IP and classifies the activity as an "attempted denial-of-service."
2. **Rapid ICMP Echo Requests:** The second rule (`alert icmp any any -> $HOME_NET any (msg:"PING FLOOD DETECTION - Rapid ICMP Echo Requests";ittype:8;flow:to_server;detection_filter: track by_src, count 50, seconds 1;classtype:attempted-dos;sid:122;))` is more sensitive, designed to catch faster floods. It triggers an alert if 50 or more ICMP echo requests from a single source

are detected within just 1 second, also targeting my \$HOME_NET and classifying it as an "attempted denial-of-service."

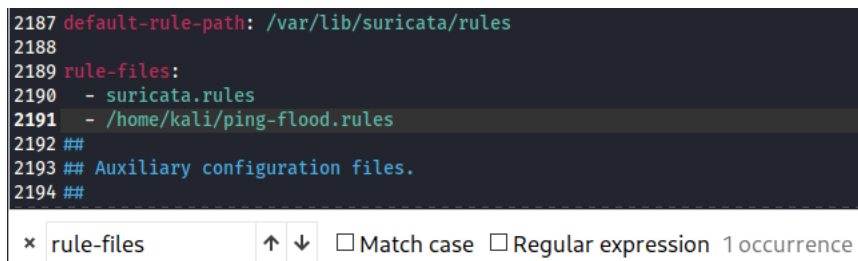
These rules are critical for demonstrating how Suricata can effectively identify and alert on malicious network activity like a ping flood, providing a clear example of an IDS in action.

Following the setup of my simulated attack, I moved to configure Suricata, my Intrusion Detection System, to properly identify and alert on such activity.



```
(kali@hemanth)-[~]  
$ sudo mousepad /etc/suricata/suricata.yaml  
[sudo] password for kali: █
```

Figure 8: sudo mousepad /etc/suricata/suricata.yaml



```
2187 default-rule-path: /var/lib/suricata/rules  
2188  
2189 rule-files:  
2190 - suricata.rules  
2191 - /home/kali/ping-flood.rules  
2192 ##  
2193 ## Auxiliary configuration files.  
2194 ##
```

× rule-files ↑ ↓ ☐ Match case ☐ Regular expression 1 occurrence

Figure 9: Updated rule-files

```

1 %YAML 1.1
2 ---
3
4 # Suricata configuration file. In addition to the comments describing all
5 # options in this file, full documentation can be found at:
6 # https://docs.suricata.io/en/latest/configuration/suricata-yaml.html
7
8 # This configuration file generated by Suricata 7.0.10.
9 suricata-version: "7.0"
10
11 ##
12 ## Step 1: Inform Suricata about your network
13 ##
14
15 vars:
16   # more specific is better for alert accuracy and performance
17   address-groups:
18     HOME_NET: "[192.168.29.0/24]"
19     HOME_NET: "[192.168.0.0/16]"
20     HOME_NET: "[10.0.0.0/8]"
21     HOME_NET: "[172.16.0.0/12]"
22     HOME_NET: "any"
23
24   EXTERNAL_NET: "!$HOME_NET"
25   #EXTERNAL_NET: "any"
26
27   HTTP_SERVERS: "$HOME_NET"
28   SMTP_SERVERS: "$HOME_NET"
29   SQL_SERVERS: "$HOME_NET"
30   DNS_SERVERS: "$HOME_NET"

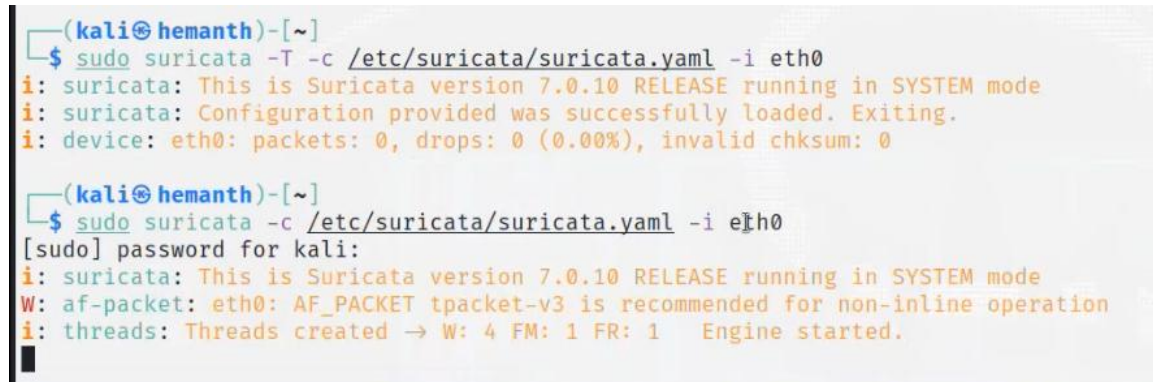
```

Figure 10: Updated Home_NET

Firstly, I opened the main Suricata configuration file, `suricata.yaml`, using the command `sudo mousepad /etc/suricata/suricata.yaml`. Within this file, I made crucial adjustments to define my internal network. Specifically, under the `address-groups` section, I set my `$HOME_NET` to include multiple common private IP ranges: `192.168.29.0/24`, `192.168.0.0/16`, `10.0.0.0/8`, and `172.16.0.0/12`. This step is essential because it tells Suricata which network segments it should consider "home" and protect, allowing it to correctly interpret the direction and context of network traffic, especially for rules like the ping flood detection ones I created that target `$HOME_NET`.

Secondly, to ensure Suricata would load and utilize my custom detection logic, I also modified the `rule-files` section within `suricata.yaml`. I added my custom `ping-flood.rules` file to this list, specifically including its full path: `/home/kali/ping-flood.rules`. This addition instructs Suricata to load my bespoke ping flood detection rules alongside its default rule sets, enabling it to actively monitor for the "Excessive ICMP Echo Requests" and "Rapid ICMP Echo Requests" that I defined earlier. These configurations are vital as they directly enable Suricata to apply the necessary rules to the network traffic and detect the ping flood generated by my Python script.

4.5 Testing and Activating Suricata

A terminal window showing two commands and their outputs. The first command is 'sudo suricata -T -c /etc/suricata/suricata.yaml -i eth0', which outputs status information about Suricata version 7.0.10 and confirms the configuration was successfully loaded. The second command is 'sudo suricata -c /etc/suricata/suricata.yaml -i eth0', which prompts for a password and then outputs a warning about AF_PACKET and a confirmation that the engine has started.

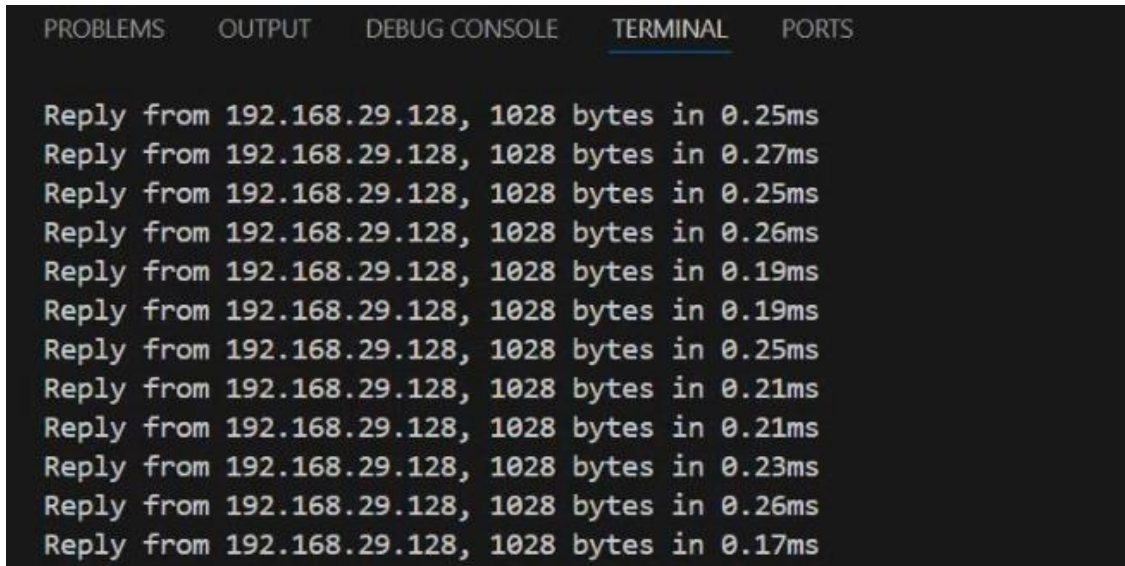
```
(kali@hemanth)-[~]  
$ sudo suricata -T -c /etc/suricata/suricata.yaml -i eth0  
i: suricata: This is Suricata version 7.0.10 RELEASE running in SYSTEM mode  
i: suricata: Configuration provided was successfully loaded. Exiting.  
i: device: eth0: packets: 0, drops: 0 (0.00%), invalid chksum: 0  
  
(kali@hemanth)-[~]  
$ sudo suricata -c /etc/suricata/suricata.yaml -i eth0  
[sudo] password for kali:  
i: suricata: This is Suricata version 7.0.10 RELEASE running in SYSTEM mode  
W: af-packet: eth0: AF_PACKET tpacket-v3 is recommended for non-inline operation  
i: threads: Threads created → W: 4 FM: 1 FR: 1    Engine started.
```

Figure 11: Testing and Activating Suricata

Before completely deploying Suricata, I did a vital test of its settings. I ran the command `sudo suricata -T -c /etc/suricata/suricata.yaml -i eth0`. The `-T` flag above tells Suricata to just test the configuration file located at `/etc/suricata/suricata.yaml` and verify it can correctly load with the `eth0` network device. The report confirmed that my Suricata version 7.0.10 was running in "SYSTEM MODE" and, more importantly, that the "Configuration provided was successfully loaded." This preliminary test ensured that there were no syntax mistakes or issues with the rule files I had just written.

After a successful configuration test, I proceeded to activate Suricata in live monitoring mode. I used a similar command, `sudo suricata -c /etc/suricata/suricata.yaml -i eth0`, but this time without the `-T` flag. This command launched Suricata, telling it to start actively inspecting traffic on my `eth0` network interface using the specified configuration. Although there was a warning about `AF_PACKET` for non-inline operation, the crucial message "Engine started" confirmed that Suricata was now actively running and monitoring my network for any activity that matches the rules I've defined, including my custom ping flood detection rules.

4.6 Initiating the Ping Flood (Attack Simulation for Detection)



```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS

Reply from 192.168.29.128, 1028 bytes in 0.25ms
Reply from 192.168.29.128, 1028 bytes in 0.27ms
Reply from 192.168.29.128, 1028 bytes in 0.25ms
Reply from 192.168.29.128, 1028 bytes in 0.26ms
Reply from 192.168.29.128, 1028 bytes in 0.19ms
Reply from 192.168.29.128, 1028 bytes in 0.19ms
Reply from 192.168.29.128, 1028 bytes in 0.25ms
Reply from 192.168.29.128, 1028 bytes in 0.21ms
Reply from 192.168.29.128, 1028 bytes in 0.21ms
Reply from 192.168.29.128, 1028 bytes in 0.23ms
Reply from 192.168.29.128, 1028 bytes in 0.26ms
Reply from 192.168.29.128, 1028 bytes in 0.17ms
```

Figure 11: Ping flood from ping_flood.py

With Suricata now actively monitoring my network traffic, I ran the ping_flood.py script, my terminal was immediately flooded with "Reply from 192.168.29.128, 1028 bytes in X ms" messages. This continuous stream of replies confirms that my script is successfully sending a high volume of ICMP echo requests and receiving responses, effectively creating the simulated ping flood. The repeated replies from 192.168.29.128 (which I'm assuming is the target within my \$HOME_NET) indicate that the attack simulation is fully underway and should now be triggering the custom ping flood detection rules I configured in Suricata. This visual feedback confirms that the "attack" is indeed being generated as intended, allowing me to observe Suricata's detection capabilities in real-time.

4.7 Successful Ping Flood Detection by Suricata

```

kali@hemanth: ~
File Actions Edit View Help
. yaml
assifigation: Attempted Denial of Service] [Priority: 2] {ICMP} 192.168.29.1:8 → 192.168.29.
07/28/2025-07:48:41.417489 [**] [1:122:0] PING FLOOD DETECTION - Rapid ICMP Echo Requests [*
assifigation: Attempted Denial of Service] [Priority: 2] {ICMP} 192.168.29.1:8 → 192.168.29.
07/28/2025-07:48:41.418382 [**] [1:122:0] PING FLOOD DETECTION - Rapid ICMP Echo Requests [*
assifigation: Attempted Denial of Service] [Priority: 2] {ICMP} 192.168.29.1:8 → 192.168.29.
07/28/2025-07:48:41.419511 [**] [1:122:0] PING FLOOD DETECTION - Rapid ICMP Echo Requests [*
. yaml
assifigation: Attempted Denial of Service] [Priority: 2] {ICMP} 192.168.29.1:8 → 192.168.29.
07/28/2025-07:48:41.420559 [**] [1:122:0] PING FLOOD DETECTION - Rapid ICMP Echo Requests [*
assifigation: Attempted Denial of Service] [Priority: 2] {ICMP} 192.168.29.1:8 → 192.168.29.
07/28/2025-07:48:41.421831 [**] [1:122:0] PING FLOOD DETECTION - Rapid ICMP Echo Requests [*
ricata.yaml -
0.10 RELEASE
successfully
.00%), invali
assifigation: Attempted Denial of Service] [Priority: 2] {ICMP} 192.168.29.1:8 → 192.168.29.
07/28/2025-07:48:41.423064 [**] [1:122:0] PING FLOOD DETECTION - Rapid ICMP Echo Requests [*
assifigation: Attempted Denial of Service] [Priority: 2] {ICMP} 192.168.29.1:8 → 192.168.29.
07/28/2025-07:48:41.424242 [**] [1:122:0] PING FLOOD DETECTION - Rapid ICMP Echo Requests [*
ata.yaml -i e
assifigation: Attempted Denial of Service] [Priority: 2] {ICMP} 192.168.29.1:8 → 192.168.29.
07/28/2025-07:48:41.425521 [**] [1:122:0] PING FLOOD DETECTION - Rapid ICMP Echo Requests [*
assifigation: Attempted Denial of Service] [Priority: 2] {ICMP} 192.168.29.1:8 → 192.168.29.
07/28/2025-07:48:41.426399 [**] [1:122:0] PING FLOOD DETECTION - Rapid ICMP Echo Requests [*
0.10 RELEASE
is recommend
1 FR: 1 Eng
assifigation: Attempted Denial of Service] [Priority: 2] {ICMP} 192.168.29.1:8 → 192.168.29.
07/28/2025-07:48:41.427211 [**] [1:122:0] PING FLOOD DETECTION - Rapid ICMP Echo Requests [*
assifigation: Attempted Denial of Service] [Priority: 2] {ICMP} 192.168.29.1:8 → 192.168.29.

```

Figure 12: Successful ping flood detection

Finally, this is the culmination of all my Suricata configuration efforts. It clearly shows the output from Suricata's monitoring, indicating that it has successfully detected the ping flood initiated by my Python script. The log entries are filled with alerts classified as "Attempted Denial of Service" and specifically "PING FLOOD DETECTION - Rapid ICMP Echo Requests." Each alert provides details like the timestamp and identifies the source IP (192.168.29.1) and destination within my network. This continuous stream of alerts, corresponding directly to the ongoing ping flood, confirms that my custom Suricata rules (for rapid ICMP detection, sid:122) are accurately triggered as intended. This outcome unequivocally demonstrates that my Suricata intrusion detection system is effectively protecting my network by identifying and alerting such malicious traffic patterns.

5.0 Conclusion

Intrusion Detection Systems like Suricata are essential components of modern cybersecurity strategies. In a world where threats grow increasingly sophisticated, passive traffic monitoring with high-speed, signature-based engines allows organizations to stay ahead of intrusions. Suricata's open-source architecture, multi-threading support, and compatibility with Snort rules make it ideal for use on penetration testing platforms like Kali Linux.

However, effective use of Suricata demands attention to detail, system tuning, regular updates, alert management, and multi-layer integration. Encrypted traffic, false positives, and performance issues must be actively managed. By following best practices such as combining IDS with firewalls and SIEM tools, using updated rules, and monitoring logs organizations can maximize the potential of Suricata.

Ultimately, Suricata's real power lies in its flexibility and visibility. Whether deployed in a government, academic, or enterprise setting, it remains a valuable tool in identifying threats before they cause serious damage. With ongoing monitoring, careful tuning, and integration with modern threat intelligence, Suricata stands as a robust choice for intrusion detection on Kali Linux.

6.0 References

Ali, W., Habib, M., Ullah, I., & Islam, M. (2021). Performance Evaluation of Suricata IDS. *International Journal of Computer Applications*, 183(23), 1–7. <https://doi.org/10.5120/ijca2021921611>

Cisco. (2025). What is network security? Retrieved from <https://www.cisco.com/site/us/en/learn/topics/security/what-is-network-security.html>

IBM Security. (2024). Escalating Data Breach Disruption Report. Retrieved from <https://newsroom.ibm.com/2024-07-30-ibm-report-escalating-data-breach>

Korczynski, M., Nosyk, Y., Lone, Q., Skwarek, M., Jonglez, B., & Duda, A. (2020). Don't forget to lock the front door! Inferring the deployment of source address validation of inbound traffic. *arXiv preprint arXiv:2002.00441*. <https://arxiv.org/abs/2002.00441>

OISF. (2024). Suricata Documentation. Retrieved from <https://docs.suricata.io/>

SANS Institute. (2023). 2023 Network Security Survey. Retrieved from <https://www.sans.org/white-papers/2023-network-security-survey/>

Santos, R., de Moura, D., & Oliveira, L. (2022). Enhancing IDS Accuracy Through Hybrid Deep Learning Models. *Computers & Security*, 112, 102536. <https://doi.org/10.1016/j.cose.2021.102536>

European Commission. (2018). General Data Protection Regulation (GDPR). https://commission.europa.eu/law/law-topic/data-protection/eu-data-protection-rules_en

Fortinet. (2025). Global Threat Landscape Report 2025. <https://www.fortinet.com/content/dam/fortinet/assets/threat-reports/threat-landscape-report-2025.pdf>

IBM Security. (2023). Cost of a Data Breach Report. <https://www.ibm.com/reports/data-breach>

OISF. (2024). Suricata Official Documentation. <https://docs.suricata.io/>

Pluralsight. (2023). Intrusion Detection Systems Explained. <https://www.pluralsight.com/tech-blog/intrusion-detection-systems>

Ali, M., Hossain, S., & Sharmin, M. (2022). Intrusion Detection System Performance Based on Sensor Placement and Rule Updates. *International Journal of Cybersecurity Intelligence and Cybercrime*, 5(2), 53–65. <https://doi.org/10.4018/IJCIC.2022070104>

Choudhury, A., & Lee, Y. (2023). Designing Enterprise-Specific IDS Policies: A Context-Aware Approach. *Journal of Information Security Research*, 12(1), 42–58.

CyberEdge. (2024). 2024 Cyberthreat Defense Report. Retrieved from <https://cyberedgereport.com/>

Gartner. (2023). Security Operations Effectiveness Survey: SIEM + IDS = Faster Response. Retrieved from <https://www.gartner.com/en/documents/security-ops-2023>

Khan, I., Arif, S., & Rehman, A. (2023). Adaptive IDS Tuning for High-Traffic Networks Using Machine Learning. *Journal of Network Security*, 19(3), 134–148.

Mahmood, R., Zhang, L., & Patel, K. (2022). Best Practices in IDS Deployment and Network Visibility. *IEEE Access*, 10, 88115–88126. <https://doi.org/10.1109/ACCESS.2022.3194588>